

Solution Architecture – ComicCraft

ComicCraft – AI Comic Story Creator using Gemini Models is organized as a layered workflow in which the user provides a story idea through the web interface, the Flask backend manages the request, Gemini models process the prompt and generate comic content, and the generated story and illustrations are returned to the user.

1. Architecture Overview

Layer / Component	Role in ComicCraft
User	Provides the initial story idea and interacts with the application.
Frontend	HTML, CSS and JavaScript provide the web interface and collect user input.
Flask Backend	Python-based application layer that receives requests, controls the workflow and communicates with AI services.
Gemini API	Processes prompts and supports AI-assisted generation of comic story content.
AI Content Generation	Generates characters, scenes, dialogues and structured comic content from the story idea.
Image Generation	Produces visual illustrations associated with the generated comic content.
Output Layer	Presents the generated comic story and illustrations back to the user.

2. End-to-End Data Flow

1. User enters a comic story idea through the web interface.
2. The frontend sends the input to the Flask backend.
3. Flask validates and manages the request and prepares the AI prompt.
4. The Gemini API receives the prompt for AI-assisted story generation.
5. Gemini-generated content is organized into characters, scenes and dialogues.
6. Image generation is used to create visual illustrations for the comic.
7. The backend prepares the generated content and visuals as the final comic output.
8. The output is displayed to the user through the frontend.

3. Technology Mapping

Technology	Architecture Responsibility
HTML / CSS / JavaScript	User interface and interaction.
Python	Backend programming and application logic.

Flask	Web framework connecting frontend requests, backend logic and AI services.
Google Gemini API / Gemini Models	AI-based processing and comic content generation.
AI Image Generation	Visual illustration generation.
Environment Variables (.env)	Configuration and secure runtime handling of API-related settings.
Git / GitHub	Version control and project source management.

4. Component Interaction

Frontend ↔ Flask Backend: User input is submitted to the backend and generated results are returned for display.

Flask Backend ↔ Gemini API: The backend prepares and sends AI requests, then receives generated content for further processing.

Backend ↔ Image Generation: Generated comic requirements are used to produce the visual portion of the comic.

Backend → Output: Story content and illustrations are combined into a user-facing comic result.

5. Architecture Objective

The architecture separates user interaction, backend processing, AI generation and output presentation. This supports a clear workflow for converting a simple story idea into structured comic content with characters, scenes, dialogues and illustrations.